Modèles Algorithmiques Fondamentaux et Mécanismes d'Accès (R1.01)

Thème principal : Initiation au développement (C++) - Algorithmique

Synthèse exécutive : Ce cours pose les fondements méthodologiques indispensables de la conception algorithmique en BUT 1 Informatique. Avant toute écriture de code, l'informaticien doit analyser le contrat d'interface et le mécanisme d'accès de sa structure de données (Direct, Séquentiel ou Dynamique). Le cours formalise trois grands modèles algorithmiques canoniques réutilisables sur des structures à accès direct : (1) Le parcours complet avec traitement systématique, (2) Le parcours complet avec traitement conditionné, et (3) La recherche de première occurrence (avec ses deux variantes). Une rigueur absolue est exigée sur le respect des bornes d'indices (de 0 à N-1), sur l'adoption du principe du pessimisme algorithmique, ainsi que sur l'obligation d'intégrer une double condition d'arrêt pour tout algorithme de recherche.

Typologie et Analyse Préalable des Mécanismes d'Accès aux Données

L'étape zéro de toute démarche d'analyse algorithmique consiste à s'interroger sur la nature des accès autorisés par la structure de données sous-jacente. Si l'on manipule une structure à accès direct (un tableau statique de N éléments ou une chaîne de caractères), l'adresse mémoire de n'importe quel élément situé à l'indice 'i' est calculable immédiatement en temps constant O(1). En revanche, sur un flux séquentiel (tel qu'un fichier CSV lu ligne par ligne), l'ordinateur ne propose que trois opérations fondamentales : se positionner au début du flux, lire l'élément suivant, et tester si la fin du flux est atteinte. Il est physiquement et logiquement impossible de sauter directement à la 50ème ligne sans avoir lu les 49 premières. Appliquer une notation par indice de type T[i] sur un fichier séquentiel constitue donc un contre-sens algorithmique qui invalide entièrement la solution proposée.

- Accès direct : Calcul immédiat de l'adresse mémoire en O(1) pour les tableaux et chaînes.
- Accès séquentiel : Navigation séquentielle par curseur (fichiers texte, CSV), sans saut d'indice direct.
- Contrat d'interface : Analyse préalable indispensable avant d'écrire la moindre ligne de code.

Cas concret enseignant : *Le professeur insiste lourdement sur un piège récurrent en évaluation : rédiger un algorithme qui utilise un accès par indice 'T[i]' pour traiter un fichier texte ou un fichier CSV brut. Comme le système d'exploitation ne fournit pour ces fichiers qu'un curseur séquentiel avance-ligne, l'algorithme est sémantiquement faux et conduit directement à la note zéro sur la question.*

Premier Modèle : Le Parcours Complet avec Traitement Systématique

Ce modèle s'applique lorsque chaque élément d'une zone définie d'une collection doit subir une opération uniforme sans aucune exception ni filtrage (exemples : initialiser toutes les cases d'un tableau à zéro, afficher le contenu complet d'un tableau, ou convertir l'ensemble des caractères d'une chaîne en majuscules). Comme la taille du domaine à parcourir est fixée dès le départ, la structure itérative appropriée est la boucle bornée 'Pour'. En notation standard, pour un tableau de N cases, la première se situe à l'indice 0 et la dernière à l'indice N - 1. Le traitement est exécuté de manière indifférenciée et systématique à chaque tour de boucle.

- Exhaustivité : Inspection de 100% des éléments de la collection.
- Boucle bornée : Utilisation obligatoire d'une structure Pour (For).
- Bornes d'indices : Indexation standard de 0 à N - 1 pour éviter le dépassement.

Cas concret enseignant : *Exemple d'illustration vu en amphithéâtre : 'Afficher l'intégralité d'un tableau de N cases'. L'enseignant rappelle que l'erreur la plus fréquente chez les étudiants débutants consiste à écrire 'Pour i allant de 0 à N', ce qui provoque un crash immédiat ('out of bounds') lors de la tentative de lecture de la case mémoire T[N] qui n'appartient pas au tableau.*

Second Modèle : Le Parcours Complet avec Traitement Conditionnel

Le parcours complet avec traitement conditionnel conserve l'exploration exhaustive du modèle 1, mais encadre l'action de traitement par une condition de filtrage. Il est utilisé pour compter des occurrences particulières (ex: compter le nombre de voyelles dans un texte), afficher uniquement un sous-ensemble de données (ex: imprimer les nombres pairs d'un tableau), ou effectuer des calculs cumulatifs ciblés. Si la condition compare un élément à son voisin précédent (T[i] > T[i-1]), le concepteur doit modifier la borne initiale (démarrer à i = 1 au lieu de 0) sous peine d'évaluer T[-1] qui provoque une erreur mémoire.

- Filtrage : Imbrication d'une structure alternative (Si) dans une boucle bornée.
- Inspection globale : Lecture de tous les éléments mais application ciblée du traitement.
- Ajustement des bornes : Attention aux comparaisons avec les voisins (i-1 ou i+1).

Cas concret enseignant : *L'analogie de la chaîne de montage en usine donnée par le professeur : Sur une chaîne de fabrication, chaque pièce défile obligatoirement devant l'inspecteur qualité (parcours complet d'inspection de toutes les pièces), mais l'inspecteur n'appose le coup de tampon de rebut (action de traitement) que si et seulement si la pièce présente un défaut spécifique (traitement conditionné).*

Troisième Modèle : La Recherche de Première Occurrence & Ses Variantes

À l'inverse des parcours complets, la recherche de première occurrence s'interrompt dès que le premier élément satisfaisant le critère est localisé. L'esprit de conception repose sur le principe du pessimisme : on initialise un booléen 'trouve' à 'Faux' avant d'engager l'exploration. Tout algorithme de recherche doit obligatoirement intégrer deux conditions d'arrêt : (1) le succès de la recherche, (2) l'épuisement de la collection. La variante 1 utilise une sortie anticipée, tandis que la variante 2 utilise une boucle 'Tant que' globale en tête de boucle.

- Pessimisme algorithmique : Initialisation de la variable 'trouve' à Faux.
- Double condition d'arrêt : Succès de la recherche OU épuisement de la collection.
- Boucle non bornée : Utilisation d'une structure Tant que (While).

Cas concret enseignant : *La règle d'or du professeur : 'En informatique, soyez pessimistes ! Partez toujours du principe que ce que vous cherchez n'existe pas dans la structure, et ne basculez votre indicateur à Vrai qu'au moment précis où vous le touchez.' Il martèle qu'un algorithme de recherche présenté en examen avec une seule condition d'arrêt recevra systématiquement la note zéro.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut appliquer le parcours complet avec traitement systématique sur une structure à accès dynamique ?
Réponse du professeur : Oui, du moment que la structure autorise l'accès itératif à l'ensemble de ses éléments, mais la structure de contrôle (boucle) doit s'adapter à la nature dynamique (par exemple une boucle Tant que basée sur l'existence d'un pointeur suivant plutôt qu'une boucle Pour bornée).

Question étudiant : Pourquoi s'embêter à nous enseigner deux variantes différentes pour la recherche de première occurrence ?
Réponse du professeur : Parce que selon le langage cible (C++ vs Python) et selon le besoin de récupérer l'indice exact de l'élément trouvé pour un traitement ultérieur, l'une des variantes sera beaucoup plus propre et sécurisée que l'autre.

Question étudiant : Comment fait-on si la condition dans un parcours complet conditionnel dépend de l'élément précédent (par exemple T[i] > T[i-1]) ?
Réponse du professeur : Il faut impérativement décaler la borne inférieure de la boucle Pour à i = 1 au lieu de i = 0, sinon la première itération évaluera T[0] > T[-1], ce qui provoque une erreur de dépassement mémoire (out of bounds).

Question étudiant : Si l'on veut récupérer TOUS les indices correspondant à une valeur donnée (par exemple trouver tous les '5' d'un tableau), quel modèle doit-on utiliser ?
Réponse du professeur : Il ne faut surtout pas utiliser le modèle 3 (recherche de première occurrence qui s'arrête au premier), mais bien utiliser le Modèle 2 (Parcours complet avec traitement conditionnel) en affichant ou stockant l'indice à chaque fois que la condition T[i] == 5 est vérifiée.

■ Exercice & Démonstration d'Application

Énoncé : Écrire en pseudo-code algorithmique rigoureux l'algorithme permettant d'afficher tous les nombres strictement positifs d'un tableau d'entiers 'T' contenant 'N' éléments. Expliciter la démarche, le modèle retenu et la gestion des bornes.

Démarche de résolution :
Étape 1 : Analyse du problème et sélection du modèle. Il faut inspecter l'intégralité des cases d'un tableau (structure à accès direct) de la première à la dernière. L'action d'affichage ne doit s'exécuter que si la valeur respecte le critère 'strictement supérieur à 0'. Le modèle algorithmique approprié est donc le 'Parcours complet avec traitement conditionné' (Modèle 2). Étape 2 : Détermination des bornes d'indices. En convention informatique standard (indexation à zéro), pour un tableau de N éléments, la première case est à l'indice 0 et la dernière à l'indice N - 1. Borne initiale = 0, Borne finale = N - 1. Étape 3 : Choix de la structure de contrôle. Le nombre d'itérations est connu avant d'entrer dans la boucle -> Utilisation d'une boucle bornée 'Pour i allant de 0 à N - 1'. Étape 4 : Écriture du filtre conditionnel et du traitement. À chaque itération i, évaluer 'Si T[i] > 0 Alors' puis déclencher l'action 'Afficher(T[i])'.

```
// CONSTANTES
// (aucune constante spécifique requise)

// VARIABLES
// (clavier) >> saisir >> N
// Entier i
// Tableau T de taille N

// TRAITEMENTS
Pour i allant de 0 à N - 1 Faire
{
    Si T[i] > 0 Alors
    {
        Afficher(T[i])
    }
Fin Si
Fin Pour
```

Résultat : Algorithme validé garantissant un parcours exhaustif sans risque de dépassement de capacité (l'itération s'arrête exactement à N-1) et appliquant le filtre sélectif demandé de manière optimale.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le risque critique si l'on écrit une boucle 'Pour i allant de 0 à N' sur un tableau de N éléments indicé à partir de 0 ?

R : Une erreur de dépassement de capacité (out of bounds) lors de l'accès à la case T[N] qui n'existe pas.

Q : Combien de conditions d'arrêt minimales doit comporter la structure itérative d'une recherche de première occurrence ?

R : Exactement deux conditions d'arrêt : le succès de la recherche ET l'épuisement de la structure.

Q : Pourquoi qualifie-t-on la démarche d'initialisation d'un algorithme de recherche de première occurrence de 'pessimiste' ?

R : Parce que l'on initialise systématiquement le booléen 'trouve' à 'Faux' en partant du principe que l'élément cherché est absent de la structure.